Thực hành 4.1 sử dụng RESTfull API

Yêu cầu nộp bài:

- Nộp file `.docx` tên: **MSSV_HovaTen_th4.1.docx**
- File phải chứa:
    1. **Ảnh chụp màn hình** kết quả chạy chương trình **cho từng chức năng**
    2. **Ảnh kiểm thử chức năng (nếu có)**

Đề bài (Cá nhân): Tạo chương trình trang quản trị_admin hoặc dựa vào bài trước (đề tài theo nhóm của bạn), sử dụng công nghệ **Spring Boot**, **RESTfull API,** mô hình MVC, lưu dữ liệu tạm trong bộ nhớ List hoặc có thể dùng CSDL. Chương trình có các chức năng CRUD. Hệ thống từ 5 bảng trở lên (Ví dụ: Product-Category-....)

Trình bày word

1. Đề tài nhóm bạn
    - Xác định danh sách bảng/thuộc tính phù hợp dự án (áp dụng kế thừa)
2. Trình bày nội dung và chụp màn hình minh chứng:
    - Kết quả chức năng CRUD cho 1 đối tượng
    - Sử dụng gì để lưu dữ liệu
    - Ý nghĩa của Annotation
    - So với bài thực hành chương 3 thì bài này có sự thay đổi như thế nào?
    - Khái niệm của @RestController? Cho hình ảnh minh chứng sử dụng trong bài
    - Chức năng của @PostMapping và @RequestMapping là gì? Cho hình ảnh minh chứng sử dụng trong bài
    - Chức năng của @PathVariable, @RequestBody là gì? Cho hình ảnh minh chứng sử dụng trong bài
    - Sử dụng HATEOAS cho 1 chức năng (chụp ảnh minh chứng, chỗ sử dụng và kết quả)

Hướng dẫn cho hai chức năng thêm và xem sản phẩm

Tóm tắt các bước

Bước 1: Tạo project Spring boot có các thư viện sau:

Bước 2: Tạo các package trong

Bước 3: model/Product.java (sử dụng lombok)

Bước 4: repository/ProductRepository.java

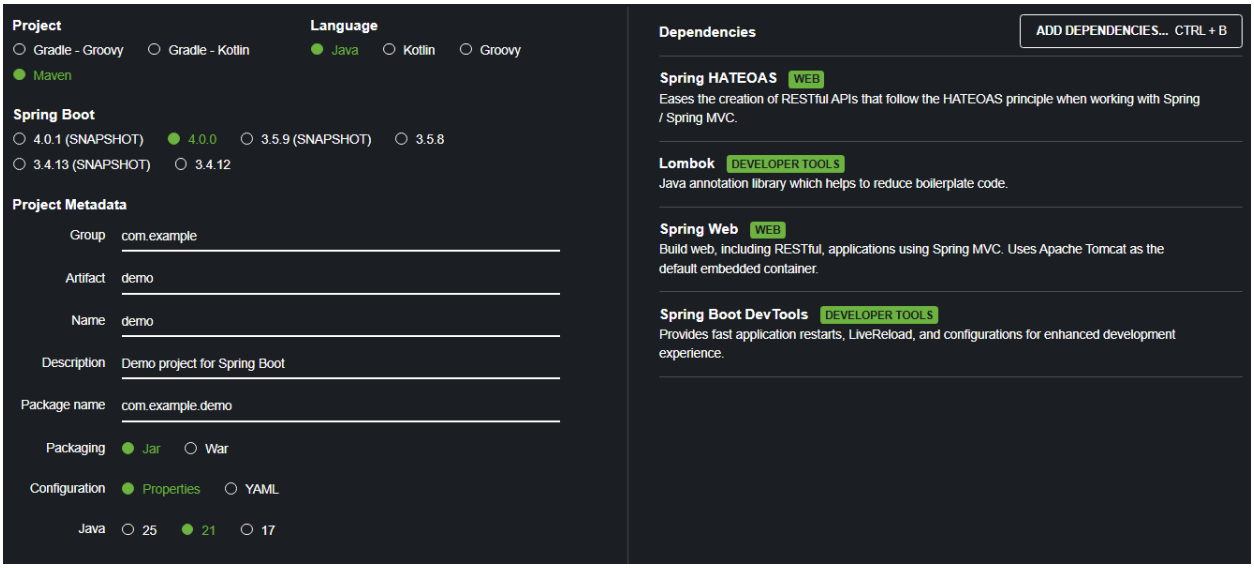
Bước 5: service/ProductService.java

Bước 6: Controller

Bước 7: Test bằng postman

Chi tiết các bước

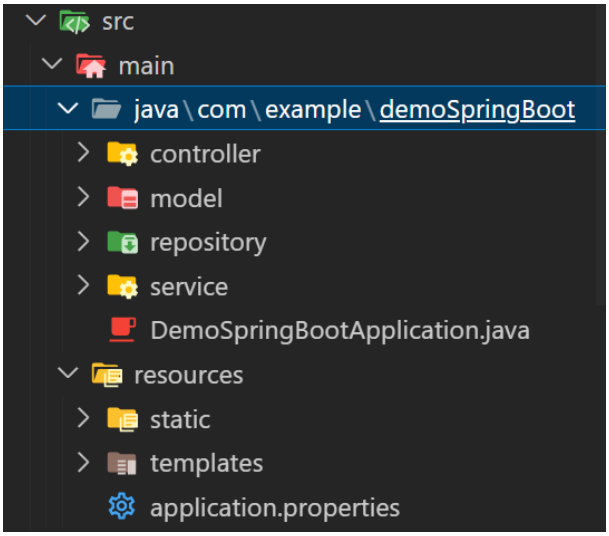
Bước 1: Tạo project Spring boot có các thư viện sau:



Cài các thư viện Dependencies:

- Spring Web
- Lombok
- Spring Boot DevTools
- HATEOAS

Bước 2: Tạo các package



Bước 3: model/Product.java (sử dụng lombok)

```
package model;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

// Lombok tự sinh getter, setter, toString, equals, hashCode
@Data
@NoArgsConstructor      // Tạo constructor rỗng
@AllArgsConstructor     // Tạo constructor đầy đủ tham số
// Lớp Product đại diện cho dữ liệu sản phẩm (Model trong MVC)

public class Product {

// Thuộc tính của sản phẩm
    private int id;
    private String name;
    private String description;
    private double price;
    private int quantity;

}
```

Bước 4: repository/ProductRepository.java

```
package repository;

//import thư viện
import java.util.List;

// Repository: Quản lý dữ liệu sản phẩm
public class ProductRepository {
```

```
    // List lưu trữ sản phẩm
    private List<Product> products = new ArrayList<>();

    // Tạo phương thức Lấy toàn bộ danh sách sản phẩm


    // Tạo phương thức Lưu một sản phẩm mới vào danh sách

}
```

Bước 5: service/ProductService.java

```
package service;

//import thư viện

// Service: Xử lý logic nghiệp vụ sản phẩm
public class ProductService {

// Gọi Repository để lưu trữ và lấy dữ liệu
    private ProductRepository repository = new ProductRepository();

   // Tạo phương thức Lấy toàn bộ danh sách sản phẩm

    // Tạo phương thức thêm sản phẩm mới

}
```

Bước 6: controller

```
package controller;


import model.Product;

import service.ProductService;

import org.springframework.web.bind.annotation.*;


import java.util.List;


@RestController

@RequestMapping("/api/products")  // API chính

public class ProductController {


    private ProductService service = new ProductService();
```

```
    // Lấy danh sách tất cả sản phẩm (GET)
    @GetMapping
    public List<Product> getAllProducts() {
        return service.getAllProducts();
    }

    // Thêm một sản phẩm mới (POST)
    @PostMapping
    public String addProduct(@RequestBody Product product) {
        service.addProduct(product);
        return "Product added successfully!";
    }
}
```

Bước 7: Test bằng postman

Nếu bạn đặt là "/api/products"

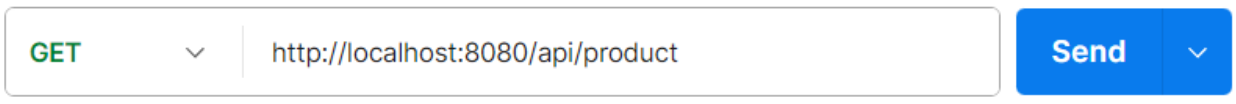
```
@RestController
@RequestMapping("/api/products")
public class ProductController {
```

Thì đường dẫn http://localhost:8080/api/product

8080 là port mà project bạn đang chạy

"/api/product" là đường dẫn để thao tác quản lý sản phẩm

Chọn **GET** để xem danh sách sản phẩm



kq

Body Cookies Headers (5) Test Results 200 OK

{} JSON Preview Visualize


```
[
    {
        "id": 1,
        "name": "Macbook Air M5",
        "description": "Apple laptop",
        "price": 100.0
    },
    {
        "id": 2,
        "name": "Macbook Air M5",
        "description": "Apple laptop",
        "price": 100.0
    },
    {
        "id": 3,
        "name": "Macbook Air M5",
        "description": "Apple laptop",
        "price": 100.0
    }
]
```

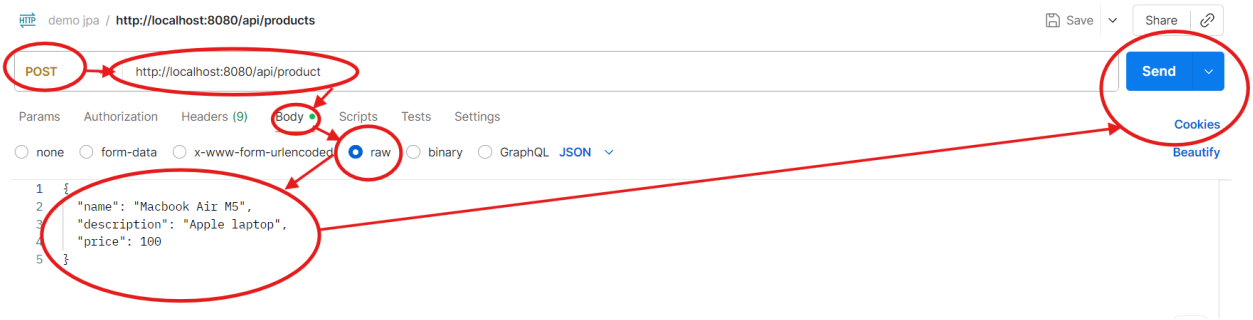

Chọn **POST** dể thêm sản phẩm

@RequestBody sẽ yêu cầu bạn nhập từ **JSON**

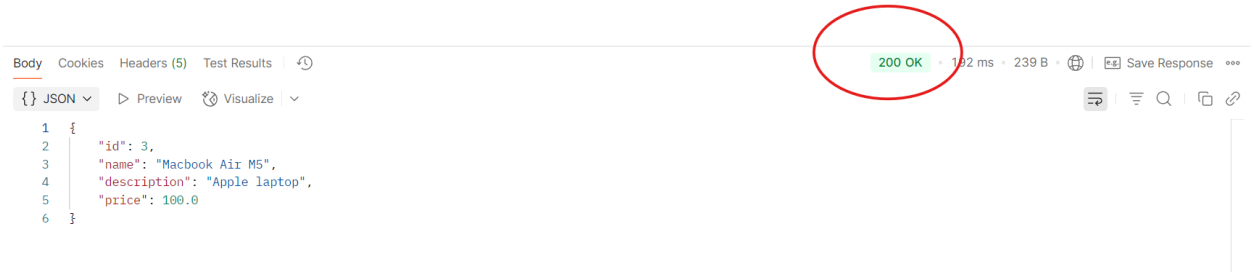
```
@PostMapping
public Product addProduct(@RequestBody Product product) {

    return productService.addProduct(product);
}
```

Bên Postman bạn sẽ sử dụng như sau



Và chúng ta có kq:



PUT /api/products/{id}: Cập nhật

DELETE /api/products/{id}: Xóa

GET /api/products/{id}: Xem chi tiết